

Removing Electrical Noise Coupling via Deep Learning Techniques

Jah Multani, Jevin Barnett, and Kyle Hildebrand
Michigan Technological University
CS5841 Final Project

Abstract—Electrical coupling in electronic control units can introduce large, irregular disturbances into low-energy measurement signals. This work evaluates compact neural networks as a software-based method for suppressing such disturbances on an embedded platform. The selected experiment uses 63,959 ordered measurements from `Book2.xlsx`, with `Sample 2` as the noisy input and `clean2=0` as the desired reference. Thirty-two-sample windows were standardized and used to train residual multilayer perceptron (MLP) and one-dimensional convolutional neural network (1D CNN) models. The MLP contained 8,385 trainable parameters and achieved a test RMSE of 4.64×10^{-4} , compared with 0.158 for the CNN and 0.278 for a 15-sample moving average. The MLP was converted from Keras to TensorFlow Lite, reducing the model from 35,796 bytes to 13,200 bytes after optimization, and a quantized forward pass executed in approximately 1.7 ms on the STM32 platform. Because the desired target is constant zero, these results demonstrate learned disturbance cancellation rather than reconstruction of an arbitrary waveform. The project nevertheless establishes a reproducible end-to-end pipeline from ECU data collection through embedded inference.

Index Terms—electrical noise, residual learning, multilayer perceptron, 1D CNN, embedded machine learning, STM32

I. INTRODUCTION

Electronic control units (ECUs) process analog measurements in electrically noisy environments. Fast switching edges, asynchronous controllers, grounding differences, and nearby power electronics can couple interference into low-energy signal paths. The resulting disturbances reduce signal integrity and may affect downstream estimation or control. Conventional remedies include shielding, grounding changes, analog filtering, and hardware redesign. These approaches can be effective, but may increase cost, size, and development effort.

Machine-learning-based correction offers a complementary software approach. A learned model can use a short history of noisy samples to estimate a correction that is applied to the current measurement. For embedded use, however, accuracy alone is insufficient: the model must also fit within limited Flash and SRAM and complete inference within the sampling interval. This project therefore compares classical smoothing with residual MLP and 1D CNN models, evaluates held-out reconstruction error, converts the selected model to TensorFlow Lite, and measures a forward pass on an STM32 microcontroller. The main contributions are: 1) an end-to-end signal-processing pipeline using real ECU measurements; 2) a comparison of compact learned and classical methods; 3) embedded conversion and timing; and 4) a critical analysis of the limits created by a constant desired reference.

II. MOTIVATION

The motivation is to determine whether a compact learned model can suppress repeatable electrical disturbances without adding analog hardware. A software correction running on an existing microcontroller may be useful when hardware changes are expensive, packaging is constrained, or the noise mechanism is difficult to model analytically. The project also investigates the accuracy-versus-cost tradeoff. A large network may perform well in Colab but be unsuitable for an ECU, whereas a smaller network may provide sufficient attenuation with predictable latency and memory usage. Accordingly, model error, parameter count, file size, and embedded runtime are considered together.

III. RELATED WORK

Donoho introduced wavelet soft-thresholding, which reconstructs a signal by transforming a noisy observation into the wavelet domain and shrinking coefficients associated with noise [1]. This provides an interpretable, non-learning baseline, although fixed thresholds may be less effective for nonstationary or structured interference. Alkhafaji *et al.* developed a lightweight 1D CNN for communication-signal classification and denoising in low-SNR edge environments [2], showing that convolutional processing can be designed with embedded latency in mind. Sun *et al.* proposed an end-to-end residual 1D CNN for artifact removal from EEG signals [3]. Their residual formulation is relevant because the model learns the corruption or correction component instead of reconstructing the complete signal independently. The present work applies these ideas to ECU measurements and extends the evaluation through TensorFlow Lite conversion and STM32 execution.

IV. METHODOLOGY

A. Data Acquisition and Problem Definition

An ECU and motor were mounted on a dynamometer. CANape was configured for consistent acquisition, selected signals were sampled at fixed rates, stored in MATLAB format, and exported to Excel. The motor was operated below 100 rpm so that back-EMF would not exceed the battery voltage and confound the measurement. Signals were selected because they exhibited electrical-noise characteristics.

The authoritative experiment in this report uses `Book2.xlsx`, containing 63,959 ordered rows and the columns `Sample 1`, `Sample 2`, `clean`, and `clean2`. Jah's notebook uses `Sample 2` as the noisy input and

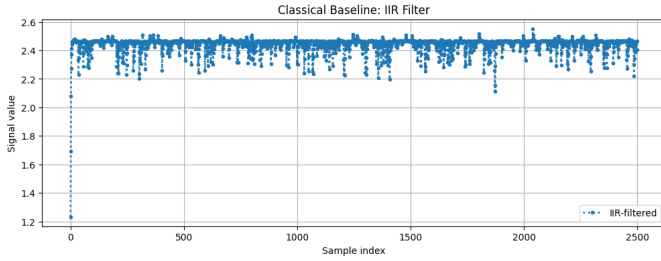


Fig. 1. IIR-filter output used as a classical denoising baseline.

`clean2` as the desired output. Both `clean` and `clean2` are identically zero, so the task is zero-reference disturbance cancellation. This distinction is important: the experiment evaluates how effectively the model drives the disturbed measurement toward a known zero reference, not whether it can reconstruct an arbitrary time-varying clean waveform.

B. Preprocessing and Feature Engineering

Column names were stripped of extra whitespace, values were converted to numeric form, and invalid rows were dropped. No rows were lost in the executed dataset. Moving averages with 5 and 15 samples were computed as classical baselines. A 32-sample sliding window was then formed,

$$\mathbf{x}_t = [x_{t-31}, \dots, x_t], \quad (1)$$

where x_t is the current noisy sample. The residual target was

$$r_t = y_t - x_t, \quad (2)$$

and the reconstructed output was

$$\hat{y}_t = x_t + \hat{r}_t. \quad (3)$$

With $y_t = 0$ in this dataset, the ideal correction is $r_t = -x_t$.

The 63,928 windows were divided chronologically into 44,749 training, 9,589 validation, and 9,590 test windows using a 70/15/15 split. Input and residual-target standardization used means and standard deviations fitted only on the training partition. The chronological split avoids random mixing of distant future and past samples. Because windows were created before splitting, the 31-sample neighborhoods at the two boundaries overlap; this limitation is discussed in Section VII.

The 15-sample moving average and IIR filter were retained as classical references. The IIR output used in the original team report is shown in Fig. 1.

C. Models, Training, and Selection

The MLP accepted a flattened 32-element window and used dense layers of 64, 64, and 32 ReLU units followed by a linear scalar output. The 1D CNN accepted a 32×1 sequence and used three convolutional stages, batch normalization, max pooling, global average pooling, a 32-unit dense layer, and a linear output. Table I summarizes the designs.

Training minimized scaled residual MSE. `ReduceLROnPlateau` halved the learning rate after ten validation epochs without improvement, and early stopping used a patience of 25 with best-weight restoration. The MLP

TABLE I
MODEL DESIGN AND TRAINING CONFIGURATION

Model	Architecture	Trainable
MLP	$32 \rightarrow 64 \rightarrow 64 \rightarrow 32 \rightarrow 1$, ReLU	8,385
1D CNN	Conv16–BN–Conv32–BN–Pool–Conv32–GAP–Dense32–1	5,921
Optimizer/loss		Adam (10^{-3}) / MSE
Batch / epochs		32 / maximum 300
Callbacks		LR patience 10; stop patience 25
Seed		NumPy and TensorFlow seed 42

continued through 300 epochs as validation loss continued to improve. The CNN stopped after 25 epochs because the same callback objects were reused after MLP training; therefore, its result is retained as an implemented comparison but treated as preliminary. The MLP was selected for deployment because it produced the lowest learned-model error, required only 8,385 parameters, and had substantially lower measured software inference time.

D. Embedded Conversion

The selected Keras MLP was converted to a 35,796-byte floating-point TensorFlow Lite file. Applying TensorFlow Lite default optimization reduced the file to 13,200 bytes. Preprocessing constants, including the 32 input means and scales and the output mean and scale, were exported with a known test vector. The model was imported into the STM32 workflow and evaluated using stored windows. A forward pass included network inference; live ADC acquisition was outside the scope of the current prototype.

V. EXPERIMENTAL RESULTS

The selected noisy signal contains disturbances ranging approximately from -10 to $+9.3$, while the desired `clean2` reference remains zero. The learned-model outputs retained from the original report are shown in Figs. 2 and 3.

The evaluation metrics were

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2, \quad (4)$$

$$\text{RMSE} = \sqrt{\text{MSE}}, \quad \text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|. \quad (5)$$

Because the target signal has zero power, conventional SNR is undefined. Instead, error attenuation was computed as

$$A_e = 20 \log_{10} \left(\frac{\text{RMSE}_{\text{raw}}}{\text{RMSE}_{\text{method}}} \right). \quad (6)$$

The MLP was the best learned model in Table II. Figure 2 shows that its output remains close to the desired reference while rejecting most large excursions. Figure 3 shows that the preliminary CNN follows substantially more of the noisy structure. The zero-reference row is intentionally included because it reveals the central limitation: a fixed zero output is perfect for this particular target.

TABLE II
BOOK2 HELD-OUT TEST RESULTS

Method	MSE	RMSE	MAE	A_e (dB)
Raw signal	9.0373×10^{-1}	0.95065	0.51153	0.00
MA, 5	1.9663×10^{-1}	0.44343	0.30017	6.62
MA, 15	7.7313×10^{-2}	0.27805	0.20303	10.68
Residual CNN	2.4934×10^{-2}	0.15790	0.11395	15.59
Residual MLP	2.1485×10^{-7}	0.000464	0.000127	66.24
Zero reference	0	0	0	—



Fig. 2. Residual MLP prediction compared with the noisy input and desired output.

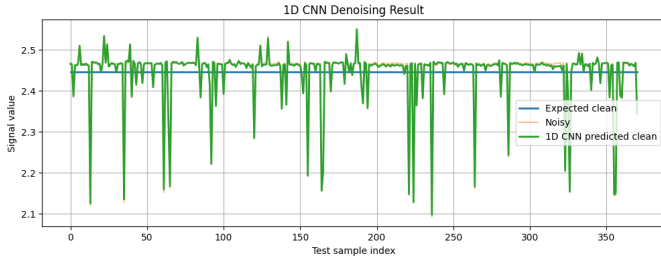


Fig. 3. Residual 1D CNN prediction compared with the noisy input and desired output.

A. Supplementary DAE and IIR Experiment

A separate team experiment compared a denoising autoencoder (DAE) with an IIR low-pass filter. That experiment is retained because it provides an additional classical-versus-learned comparison, but its statistics are not ranked directly against Table II because the preprocessing and model implementation differed. The raw signal had standard deviation 0.6449 and range $[-10.005, 9.271]$. The DAE reduced the standard deviation to 0.1834 and the range to $[-1.226, 1.345]$, while the IIR produced standard deviation 0.1822 and range $[-1.579, 1.746]$. The DAE therefore removed extreme excursions slightly more effectively, whereas the two methods had similar variance. This result supports the conclusion that compact learned models can behave comparably to conventional filtering, but it does not replace the controlled Book2 comparison.

B. Embedded Results

The 1.7 ms STM32 result demonstrates that the selected network can execute on the target platform using stored test windows. Runtime activation storage for the manual MLP buffers is below 1 kB, while parameter storage is approximately 33.5 kB in float32 form. These values are small relative to the selected STM32F407 platform and leave capacity for application code and signal buffers.

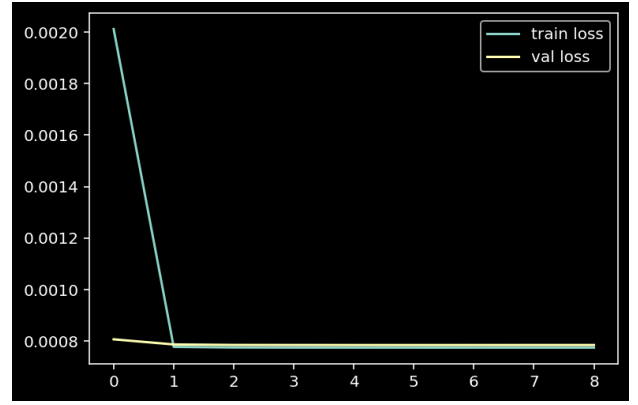


Fig. 4. Denoising-autoencoder training and validation loss.

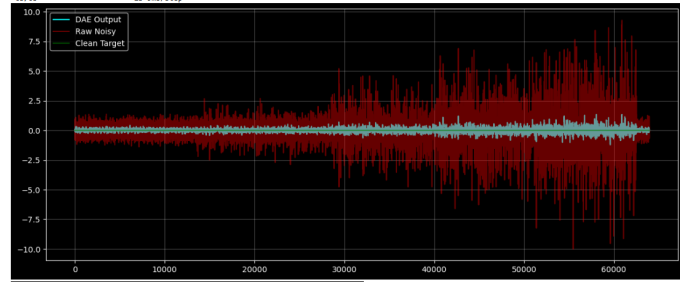


Fig. 5. Denoising-autoencoder output compared with the raw input and clean target.

	Model	Mean	Standard Dev	Var	Min	Max
0	Baseline	0.009477	0.644937	0.415944	-10.005383	9.271493
1	DAE	0.009450	0.183386	0.033630	-1.226472	1.345005
2	IIR	0.009469	0.182215	0.033202	-1.579171	1.746068

Fig. 6. Statistical comparison of the raw signal, denoising autoencoder, and IIR filter.

TABLE III
SELECTED MLP DEPLOYMENT CHARACTERISTICS

Trainable parameters	8,385
Float32 weight storage	33,540 bytes
Float TFLite file	35,796 bytes
Optimized TFLite file	13,200 bytes
STM32 forward pass	1.7 ms
Colab MLP inference	4.63 ms/window
Colab CNN inference	21.03 ms/window

VI. REPRODUCIBILITY, REPOSITORY, AND DOCUMENTATION

The complete project materials are available in the GitHub repository. It contains the final notebooks, report source, README, environment requirements, exported Keras and TensorFlow Lite models, result figures, and STM32 project or deployment files. The README documents the dataset columns, expected directory structure, team contributions, and exact execution sequence.

The Colab environment used Python 3 with TensorFlow 2.20.0, NumPy, pandas, Matplotlib, scikit-learn, and openpyxl. Reproduction consists of: 1) cloning the repository; 2) installing `requirements.txt`; 3) placing

Book2.xlsx in the documented data directory or selecting it through the notebook upload fallback; 4) running the notebook from top to bottom; and 5) comparing the regenerated metrics and model files with the committed results. Random seeds are fixed at 42. The notebook prints dataset shapes, scalar dimensions, model summaries, held-out metrics, TFLite file sizes, and a C-formatted STM32 test vector. The embedded README describes importing the model, building the project, running the stored test window, and comparing the STM32 output with the Colab reference.

VII. DISCUSSION

The MLP achieved the lowest learned-model error and the best current balance of footprint and runtime. Its 8,385 parameters correspond to only 33.5 kB of float32 weights, and the optimized TensorFlow Lite file is 13.2 kB. The CNN had fewer trainable parameters but higher error and substantially slower Colab inference. The result suggests that dense layers are sufficient for this dataset because the desired correction is dominated by the current sample rather than a complex waveform structure.

The metrics must be interpreted carefully. Since `clean2=0` for every row, the residual target is exactly $-x_t$, and a constant zero predictor obtains zero error. Thus, the MLP's 66.24 dB error attenuation demonstrates that a compact network can learn disturbance cancellation, but it does not prove general denoising or adaptation to arbitrary desired signals. A broader claim requires nonconstant hardware-corrected targets or a runtime reference supplied as an additional model input.

Three implementation limitations remain. First, windows were created before splitting, so the 31-sample neighborhoods immediately around the train/validation and validation/test boundaries overlap. A stricter pipeline should split the raw sequence first and then create windows independently. Second, the CNN reused callback objects after MLP training and stopped at epoch 25; a final comparison should initialize new callbacks for each model and select architecture using validation performance only. Third, STM32 evaluation used stored windows rather than a live ADC stream, so the reported latency does not include acquisition, buffering, preprocessing, or output communication. Future work should collect multiple operating conditions, include varying desired signals, repeat training across seeds, evaluate quantization error explicitly, and measure complete end-to-end runtime.

VIII. CONCLUSION

This project developed and documented an end-to-end workflow for suppressing coupled electrical disturbances in ECU measurements using compact neural networks. The selected Book2 experiment used 32-sample windows, residual learning, chronological partitions, training-only standardization, classical baselines, TensorFlow Lite conversion, and STM32 execution. The 8,385-parameter MLP achieved an RMSE of 4.64×10^{-4} and executed in approximately 1.7 ms on the embedded platform, outperforming the implemented moving-average and preliminary CNN alternatives.

The present result should be described as zero-reference disturbance cancellation rather than arbitrary waveform reconstruction. Even with that limitation, the work demonstrates a reproducible embedded-ML pipeline and identifies the exact data and evaluation changes required for a stronger general-denoising claim.

REFERENCES

- [1] D. L. Donoho, "De-noising by soft-thresholding," *IEEE Transactions on Information Theory*, vol. 41, no. 3, pp. 613–627, May 1995, doi: 10.1109/18.382009.
- [2] M. J. A. Alkhafaji, R. N. Mohammed, H. C. Mohammed, and M. A. N. Abed, "Lightweight 1D CNN for unified real-time communication signal classification and denoising in low-SNR edge environments," *Buletin Ilmiah Sarjana Teknik Elektro*, vol. 7, no. 3, pp. 496–508, 2025, doi: 10.12928/biste.v7i3.13789.
- [3] W. Sun, Y. Su, X. Wu, and X. Wu, "A novel end-to-end 1D-ResCNN model to remove artifact from EEG signals," *Neurocomputing*, vol. 404, pp. 108–121, Sep. 2020, doi: 10.1016/j.neucom.2020.04.029.
- [4] T. Haslwanter, *Hands-on Signal Analysis with Python: An Introduction*. Cham, Switzerland: Springer, 2021, doi: 10.1007/978-3-030-57903-6.